



Міністерство освіти і науки України
Національний технічний університет України “Київський політехнічний
інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №5
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛОНИ «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF
RESPONSIBILITY», «PROTOTYPE» »**
Система ведення нотаток

Виконав:
Студент групи ІА-24
Шкарніков А. С.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:	3
Хід роботи	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми.....	3
2. Реалізувати один з розглянутих шаблонів за обраною темою.....	5
3. Перевірка патерну	7
Висновки:	8
Код проекту:	8

Тема:

ШАБЛони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE»

Мета:

Ознайомитися з основними шаблонами проектування, такими як «Adapter», «Builder», «Command», «Chain of Responsibility», «Prototype», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Завдання:

В якості нотаток має бути, як мінімум, текст або зображення, додатково можна додати підтримку файлів розміром до 2Мб, збереження посилань на веб-сторінки, збереження html-тексту з коректним його відображенням. Система повинна дозволити вести нотатки онлайн, заводити блокноти, прикріплювати до нотаток мітки, давати можливість працювати в системі одночасно багатьом користувачам і одному користувачу працювати з різних комп'ютерів. Користувач повинен мати можливість дати доступ до свого блокноту з замітками іншому користувачу з різними правами (тільки перегляд; перегляд та редагування; перегляд, редагування, додавання та видалення нотаток)

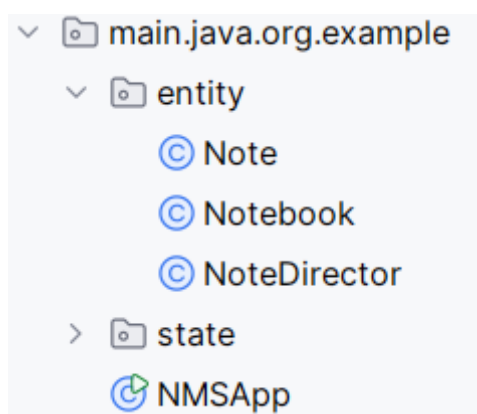
Хід роботи**1. Реалізувати не менше 3-х класів відповідно до обраної теми**

Рис. 1 — Структура проєкту

У процесі виконання лабораторної роботи було створено три класи, які забезпечують реалізацію функціоналу системи ведення нотаток (рис. 1). Наведемо детальний опис кожного з цих класів:

1. Note

Клас `Note` представляє об'єкт нотатки в системі. Він використовується для зберігання інформації, такої як текстовий вміст, шлях до зображення, прикріплений файл, HTML-контент і список тегів. `Note` є незмінним (immutable), оскільки всі його поля є фінальними, а значення задаються лише під час створення через клас `NoteBuilder`. Цей підхід гарантує, що об'єкт нотатки залишається стабільним і непридатним для змін після створення. Геттери забезпечують доступ до даних нотатки, а приватний конструктор запобігає прямому створенню екземпляра класу, забезпечуючи контроль над процесом через `Builder`.

2. `NoteBuilder`

Клас `NoteBuilder` реалізує патерн `Builder` і забезпечує поетапне створення об'єкта `Note`. Він містить набір методів, таких як `setText`, `addImage`, `attachFile`, `addHtmlContent`, `addTag`, які дозволяють задавати різні властивості нотатки. Ці методи повертають сам об'єкт `NoteBuilder`, що дозволяє будувати нотатку через ланцюжковий виклик методів. Завершується процес створення викликом методу `build`, який повертає готовий об'єкт `Note`. Цей клас дозволяє гнучко створювати нотатки з різними комбінаціями властивостей, що особливо корисно для складних сценаріїв.

3. `NoteDirector`

Клас `NoteDirector` є допоміжним компонентом, що додає зручність у створенні нотаток для типових сценаріїв. Він інкапсулює логіку побудови і використовує об'єкт `NoteBuilder` для створення нотаток за заздалегідь визначеними шаблонами, такими як текстова нотатка або нотатка з HTML-вмістом і тегами. Це полегшує використання системи, оскільки дозволяє клієнтам створювати стандартні об'єкти нотаток без необхідності вручну задавати всі властивості. `NoteDirector` забезпечує чіткий розподіл відповідальності між складним процесом побудови та спрощеним доступом до поширених шаблонів нотаток.

2. Реалізувати один з розглянутих шаблонів за обраною темою

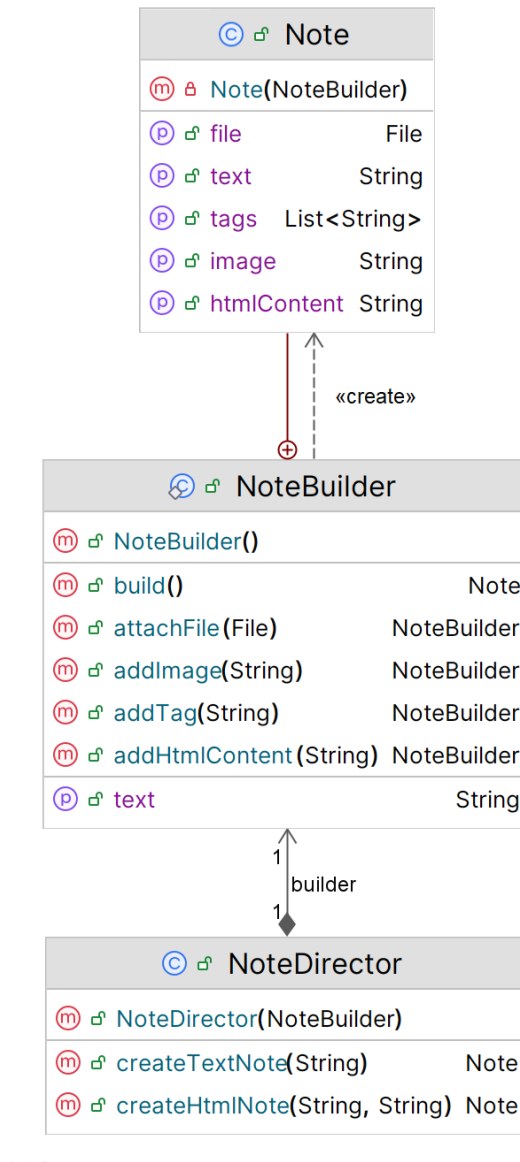


Рис. 2 — Діаграма класів

Патерн Builder був реалізований для створення об'єктів нотаток (Note). Це було необхідно, оскільки нотатки можуть мати різні властивості (текст, зображення, прикріплені файли, HTML-контент, теги), які користувач задає залежно від потреб. Створення такого об'єкта вручну із зазначенням усіх параметрів у конструкторі було б складним, особливо якщо багато з цих параметрів є необов'язковими.

Реалізація

Клас **NoteBuilder** дозволяє користувачам поетапно задавати тільки ті параметри, які їм потрібні, через набір методів, що задають різні властивості нотатки. Після завершення налаштування викликається метод `build`, який створює готовий об'єкт **Note**. Для часто вживаних шаблонів нотаток клас **NoteDirector** забезпечує зручність, пропонуючи заздалегідь визначені сценарії побудови нотаток.

Проблеми, які вирішує патерн

- **Складність і необов'язкові параметри:** Нотатка може мати багато різних властивостей, більшість з яких є необов'язковими. Builder дозволяє створювати нотатки з різними комбінаціями цих властивостей, уникаючи великої кількості конструкторів або неінформативних викликів із численними null-параметрами.
- **Гнучкість:** У разі розширення функціоналу (наприклад, додавання нових типів даних для нотаток), Builder можна легко адаптувати, додаючи нові методи.
- **Зручність і читабельність:** Завдяки ланцюжковому виклику методів створення нотаток стає зрозумілим і компактним, навіть якщо об'єкт має багато параметрів.

Переваги використання

- **Зручність для користувача:** Складні об'єкти можна створювати легко, зрозуміло і без помилок.
- **Розширюваність:** У майбутньому ми можемо додати нові властивості нотаток без значних змін у вже існуючому коді.
- **Модульність:** Чіткий поділ логіки створення (NoteBuilder) і зручного використання готових шаблонів (NoteDirector) робить код більш підтримуваним і структурованим.

3. Перевірка патерну

```
public class NMSApp {
    public static void main(String[] args) {
        Note note = new Note.NoteBuilder()
            .setText("This is a note")
            .addImage( imagePath: "path/to/image.png")
            .attachFile(new File( pathname: "path/to/file.pdf"))
            .addHtmlContent("<h1>Hello, world!</h1>")
            .addTag("Important")
            .addTag("Work")
            .build();

        System.out.println("Text: " + note.getText());
        System.out.println("Image: " + note.getImage());
        System.out.println("File: " + (note.getFile() != null ? note.getFile().getPath() : "No file attached"));
        System.out.println("HTML Content: " + note.getHtmlContent());
        System.out.println("Tags: " + String.join( delimiter: ", ", note.getTags()));

        NoteDirector director = new NoteDirector(new Note.NoteBuilder());
        Note textNote = director.createTextNote("This is a simple text note.");
        System.out.println("Text Note: " + textNote.getText());
    }
}
```

Рис. 3 — Перевірка роботи

У методі `main` демонструється створення та використання нотаток у системі ведення нотаток. Спочатку застосовується патерн Builder для створення складної нотатки. Викликаються різні методи `NoteBuilder` для поетапного налаштування властивостей нотатки, таких як текст, зображення, файл, HTML-контент і теги. Після налаштування викликається метод `build`, який повертає об'єкт `Note`. Після створення нотатки система виводить її властивості на консоль, використовуючи геттери, щоб продемонструвати результат її створення. Це показує, як Builder дозволяє гнучко створювати нотатки з різними комбінаціями параметрів.

```
Text: This is a note
Image: path/to/image.png
File: path\to\file.pdf
HTML Content: <h1>Hello, world!</h1>
Tags: Important, Work
Text Note: This is a simple text note.
```

Рис. 4 — Результат роботи

Далі, метод демонструє використання класу `NoteDirector` для створення типової текстової нотатки. `NoteDirector` отримує екземпляр `NoteBuilder` як параметр і викликає визначений шаблонний метод `createTextNote`, який налаштовує та

повертає текстову нотатку з заданим вмістом. Отримана нотатка також виводиться на консоль, демонструючи, як `NoteDirector` спрощує створення стандартних типів нотаток, приховуючи деталі їх побудови. Це забезпечує зручність і легкість використання системи для користувачів.

Висновки:

У ході лабораторної роботи було реалізовано патерн `Builder` в системі ведення нотаток, що дозволило спростити створення нотаток. Це забезпечило гнучкість і модульність коду, дозволяючи легко додавати нові функції та змінювати існуючі без значних змін у системі.

Код проекту:

https://github.com/Atl4sDev/TRPZ_labs